

# Contents

|                                                                                           |          |
|-------------------------------------------------------------------------------------------|----------|
| <b>Guide de présentation — Jour J</b>                                                     | <b>1</b> |
| TL;DR — Ce qui se passe ce jour-là . . . . .                                              | 1        |
| 1. Checklist matérielle (à préparer la veille) . . . . .                                  | 2        |
| 2. Préparation 30 minutes avant la soutenance . . . . .                                   | 2        |
| 3. Script de présentation — minute par minute (12 min) . . . . .                          | 2        |
| Slide 1 — Couverture (15 sec) . . . . .                                                   | 3        |
| Slide 2 — Plan (20 sec) . . . . .                                                         | 3        |
| Slide 3 — Le problème (1 min 15) . . . . .                                                | 3        |
| Slide 4 — Architecture (1 min) . . . . .                                                  | 3        |
| Slide 5 — PostgreSQL : 6 tables (1 min) . . . . .                                         | 3        |
| Slide 6 — Logique métier (1 min) . . . . .                                                | 4        |
| Slide 7 — Trigger phare (1 min 30) . . . . .                                              | 4        |
| Slide 8 — MongoDB : collection (1 min) . . . . .                                          | 4        |
| Slide 9 — Les 7 types de requêtes (1 min 30) . . . . .                                    | 4        |
| Slide 10 — Pipeline complet (1 min) . . . . .                                             | 5        |
| Slide 11 — Dashboard (30 sec) . . . . .                                                   | 5        |
| DÉMO EN DIRECT — 2 à 3 minutes . . . . .                                                  | 5        |
| Slide 12 — Conformité CDC (1 min) . . . . .                                               | 5        |
| Slide 13 — Difficultés et perspectives (1 min) . . . . .                                  | 6        |
| Slide 14 — Merci (15 sec) . . . . .                                                       | 6        |
| 4. Démo de secours (en cas de problème technique) . . . . .                               | 6        |
| 5. Questions probables du prof — et tes réponses . . . . .                                | 6        |
| Q1. Pourquoi avoir utilisé les deux SGBD ensemble ? Un seul ne suffit pas ? . . . . .     | 6        |
| Q2. C'est quoi la différence entre une vue et une vue matérialisée ? . . . . .            | 6        |
| Q3. Pourquoi le type INET au lieu de VARCHAR ? . . . . .                                  | 7        |
| Q4. Pourquoi avoir mis un index GIN sur le payload JSONB ? . . . . .                      | 7        |
| Q5. Comment marche ta chaîne de hashes dans journal_audit ? . . . . .                     | 7        |
| Q6. C'est quoi LISTEN / NOTIFY ? . . . . .                                                | 7        |
| Q7. Pourquoi pas SQL Server ou Oracle ? . . . . .                                         | 7        |
| Q8. Comment tu déploierais ce système en production ? . . . . .                           | 7        |
| Q9. Tu as testé avec combien de données ? . . . . .                                       | 8        |
| Q10. Si je te demande de présenter ça à un DSI non technique, comment tu fais ? . . . . . | 8        |
| 6. Erreurs à éviter absolument . . . . .                                                  | 8        |
| 7. Cheat sheet — chiffres et termes à retenir . . . . .                                   | 8        |
| 8. Ouverture — le mot de la fin . . . . .                                                 | 9        |
| Bon courage. Tu connais ton sujet. Maintenant, montre-le. . . . .                         | 9        |

## Guide de présentation — Jour J

**Projet :** SIEM (Security Information & Event Management) **Module :** Base de Données Avancée — CI1 G1 **Examineur :** Pr. Achraf Chakir BARAKA **Étudiant :** Yassir ZHD **Date :** 22 mai 2026

### TL;DR — Ce qui se passe ce jour-là

1. Tu arrives **15 minutes en avance** avec un sac contenant tout ce qui suit (cf. checklist plus bas).
2. Tu fais **10 à 12 minutes de présentation** (chronométrée) avec les 14 slides.
3. Tu fais **2 à 3 minutes de démo en direct** sur ton PC.

4. Tu réponds **5 à 10 minutes de questions** du prof.
5. **Total : ~20 minutes.**

L'objectif n'est pas d'être parfait, c'est d'être **clair, honnête** et de montrer que **tu comprends ce que tu as fait**.

---

## 1. Checklist matérielle (à préparer la veille)

À mettre dans ton sac la veille :

- ☐ **Laptop** avec :
  - ☐ Docker Desktop installé et fonctionnel
  - ☐ Le dossier `projet_siem/` à la racine du Bureau
  - ☐ Les conteneurs déjà lancés une fois (au moins testés une heure avant)
  - ☐ Le dashboard Streamlit déjà ouvert dans Chrome (onglet préchargé)
  - ☐ PgAdmin ou DBeaver configuré sur la DB (raccourci de connexion sauvegardé)
  - ☐ Mongo Express ouvert dans un onglet (`http://localhost:8081`)
  - ☐ Le PDF du rapport et le PPTX dans un dossier visible
- ☐ **Chargeur** du laptop
- ☐ **Adaptateur HDMI / VGA** au cas où la salle n'a pas le câble
- ☐ **Clé USB** avec :
  - ☐ `presentation_SIEM.pptx`
  - ☐ `rapport.pdf`
  - ☐ `projet_siem.zip` complet
  - ☐ Les screenshots du dashboard
- ☐ **Une bouteille d'eau** (la voix sèche après 10 min de parole)
- ☐ **Un stylo et une feuille** pour noter les remarques du prof

**Astuce :** la veille, tu lances `setup.ps1` une fois et tu vérifies que tout marche. Le matin du jour J, tu ne touches plus à rien.

---

## 2. Préparation 30 minutes avant la soutenance

1. Allume le laptop, branche-le sur secteur (la batterie peut lâcher).
  2. Ouvre **Docker Desktop** et attends qu'il soit "Engine running".
  3. Dans PowerShell : `cd projet_siem && docker compose up -d`. Attends 30 s.
  4. Lance le dashboard : `.\.venv\Scripts\streamlit.exe run 10_dashboard.py`.
  5. Ouvre dans Chrome **trois onglets côte à côte** :
    - Onglet 1 : `http://localhost:8501` (dashboard)
    - Onglet 2 : `http://localhost:8081` (Mongo Express)
    - Onglet 3 : `presentation_SIEM.pptx` ouvert en mode présentation
  6. Ouvre **PgAdmin** ou **DBeaver** connecté à `siem_db`, prêt à exécuter le fichier `demo.sql`.
  7. Rejoue mentalement les **trois moments clés** de la démo (cf. section 4).
- 

## 3. Script de présentation — minute par minute (12 min)

**Légende :** [ DIRE ] = ce que tu dis / [ FAIRE ] = ce que tu fais

## Slide 1 — Couverture (15 sec)

[ FAIRE ] Affiche la slide de garde.

[ DIRE ] « *Bonjour Monsieur, bonjour à tous. Aujourd'hui je vais vous présenter mon projet de Base de Données Avancée : un SIEM, c'est-à-dire un système de centralisation des alertes de cybersécurité. C'est un sujet que j'ai choisi parce qu'il correspond à ma spécialisation cybersécurité, et parce qu'il illustre parfaitement la complémentarité PostgreSQL + MongoDB.* »

## Slide 2 — Plan (20 sec)

[ FAIRE ] Slide 2.

[ DIRE ] « *Voici comment je vais organiser ma présentation. Je vais d'abord expliquer le problème réel que résout un SIEM, puis l'architecture du système. Ensuite je détaille la partie PostgreSQL — où est implémentée toute la logique métier — puis la partie MongoDB qui sert de couche d'affichage. Je termine par une démonstration en direct, la conformité au cahier des charges, et les perspectives.* »

## Slide 3 — Le problème (1 min 15)

[ FAIRE ] Slide 3.

[ DIRE ] « *Une entreprise moyenne reçoit plus de 10 000 alertes de sécurité par jour qui viennent de sources hétérogènes : firewalls, serveurs, postes utilisateurs, bases de données. Aucun analyste humain ne peut tout regarder. Le SIEM résout ce problème en centralisant toutes ces alertes, en les priorisant automatiquement avec un score de risque entre 0 et 100, et en créant des incidents agrégés quand il y a un pic d'attaques. Mon système gère 350 alertes en 7 jours, calcule le score automatiquement par trigger, et peut réagir en moins d'une seconde grâce au streaming temps réel.* »

[ ASTUCE ] **Insister** sur les chiffres : « 350 », « score 0-100 », « moins d'une seconde ».

## Slide 4 — Architecture (1 min)

[ FAIRE ] Slide 4.

[ DIRE ] « *Voici l'architecture en quatre étages. À gauche, les équipements surveillés. Au centre, **PostgreSQL 16** qui joue le rôle de source de vérité : c'est lui qui garantit l'intégrité ACID des incidents et de l'audit. À droite, **MongoDB 7** qui sert de couche d'affichage pour le dashboard, parce qu'il est plus rapide pour afficher 350 alertes sans faire 4 jointures. Entre les deux, un script Python — `psycpg2 + pymongo` — qui transforme les données. Et tout en bas, le dashboard web Streamlit. C'est une architecture classique : Splunk, Elastic Security et IBM QRadar fonctionnent toutes sur le même principe.* »

## Slide 5 — PostgreSQL : 6 tables (1 min)

[ FAIRE ] Slide 5.

[ DIRE ] « *Le cahier des charges demande au minimum 3 tables avec clés primaires et étrangères. J'en ai 6, qui couvrent toutes les notions du cours : les équipements surveillés avec une adresse IP en INET — c'est le type natif PostgreSQL pour les IP — ; les analystes du SOC avec un email hashé en SHA-256 grâce à `pgcrypto`, ce qui répond aux exigences RGPD ; les règles de détection ; les alertes elles-mêmes — c'est la table qui contient les 350 lignes — ; les incidents qui sont créés automatiquement par trigger, et le journal d'audit qui implémente une chaîne de hashes infalsifiable.* »

[ ASTUCE ] **Si le prof t'arrête sur "INET"** : « C'est un type natif PostgreSQL qui valide automatiquement le format IPv4 et IPv6, et qui supporte les opérateurs réseau comme l'inclusion dans un sous-réseau CIDR. »

## Slide 6 — Logique métier (1 min)

[ FAIRE ] Slide 6.

[ DIRE ] « Voici la grille récapitulative de tout ce que j'ai implémenté côté PostgreSQL. Le cahier des charges demande au minimum 2 séquences, j'en ai 6. Au minimum 2 fonctions PL/pgSQL, j'en ai 6. Au minimum 2 triggers, j'en ai 4. Au minimum 2 procédures avec boucles, j'en ai 5 et elles utilisent les trois types de boucles : LOOP, FOR et WHILE. Le cahier demande une vue globale, j'en ai 4 plus une vue matérialisée pour le dashboard. Et j'ai ajouté 16 index pour la performance dont un GIN sur le champ JSONB et un trigram sur le nom des équipements. »

## Slide 7 — Trigger phare (1 min 30)

[ FAIRE ] Slide 7.

[ DIRE ] « Je voudrais m'arrêter sur le trigger le plus important : celui qui illustre toute la valeur d'un SIEM. À gauche, l'analyste écrit un INSERT brut dans la table alerte, sans préciser le score. À droite, ce qui se passe automatiquement : un trigger BEFORE INSERT calcule le score selon la sévérité de la règle et l'heure de l'attaque — les attaques de nuit ajoutent 10 points. Ensuite, un pg\_notify est envoyé sur le canal "alertes\_channel" pour les scripts Python qui écoutent en streaming. Puis un trigger AFTER INSERT vérifie : s'il y a déjà 5 alertes critiques en une heure sur le même équipement, il crée automatiquement un INCIDENT agrégé. Et enfin, le trigger d'audit ajoute une ligne signée SHA-256 dans journal\_audit. Un seul INSERT, quatre actions automatiques. »

[ ASTUCE ] **Si le prof demande "pourquoi BEFORE et AFTER ?"** : « Parce que je dois calculer le score AVANT que la ligne soit écrite — c'est BEFORE —, mais je ne peux décider de créer un incident qu'APRÈS, une fois que le score est connu et que la ligne est dans la table — c'est AFTER. »

## Slide 8 — MongoDB : collection (1 min)

[ FAIRE ] Slide 8.

[ DIRE ] « Côté MongoDB, le cahier des charges demande au moins 15 champs par document. J'en ai 19. À gauche, vous voyez un exemple de document : on retrouve les champs simples — id, date, score, statut — et surtout les sous-documents source, destination, équipement, règle, et payload qui est un objet JSON imbriqué. C'est cette structure imbriquée qui permet au dashboard de tout afficher en une seule requête, sans jointure. C'est exactement la philosophie NoSQL. À droite, les chiffres : 19 champs, 350 documents, 5 index dont un sur les sous-champs JSON. »

## Slide 9 — Les 7 types de requêtes (1 min 30)

[ FAIRE ] Slide 9.

[ DIRE ] « Le cahier des charges demande sept types de requêtes différents. Je les ai toutes, regroupées dans le fichier 08\_mongodb\_requetes.js. Une condition numérique : score supérieur à 70. Une condition multiple : sévérité critique OU score supérieur à 80, avec l'opérateur \$or. Une recherche textuelle : toutes les IP qui commencent par 45 grâce à \$regex. Une mise à jour : updateMany pour marquer une IP en faux positif. Une suppression : deleteMany sur les alertes LOW. Une agrégation avec moyenne, somme et maximum par catégorie. Et une projection : on ne renvoie que les champs utiles au dashboard. »

## Slide 10 — Pipeline complet (1 min)

[ FAIRE ] Slide 10.

[ DIRE ] « Pour résumer, voici le pipeline complet en 4 étapes. Première étape : quelqu'un — un analyste ou une sonde réseau — fait un INSERT brut. Deuxième étape : les triggers détectent les seuils et créent l'incident. Troisième étape : la diffusion vers MongoDB se fait soit par batch via le script Python ETL, soit en streaming via LISTEN/NOTIFY. Quatrième étape : le dashboard Streamlit lit MongoDB et affiche les KPI, la heatmap et les top attaquants en temps réel. C'est ce que je vais vous montrer en direct juste après. »

## Slide 11 — Dashboard (30 sec)

[ FAIRE ] Slide 11.

[ DIRE ] « Voici une capture du dashboard. C'est une vraie console SOC : les KPI en haut — total events, critical, high, score moyen, incidents ouverts — la timeline des événements par sévérité, le donut de répartition, la heatmap équipement par catégorie d'attaque, et les événements récents. Comme vous le voyez, ça ressemble à ce que ferait un vrai SOC analyst en entreprise. »

[ FAIRE ] **Bascule maintenant sur Chrome onglet 1** pour la démo en direct.

## DÉMO EN DIRECT — 2 à 3 minutes

[ FAIRE ] Bascule sur le **dashboard Streamlit** dans Chrome.

[ DIRE ] « Voilà le dashboard en vrai. On voit les 379 alertes des 7 derniers jours, dont 141 critiques et 151 élevées. La timeline montre clairement un pic d'activité hier soir, qui correspond à un brute-force détecté par mes règles. La heatmap permet d'identifier que SRV-DB-01 est l'équipement le plus ciblé. Les deux incidents en bas ont été créés automatiquement par mon trigger AFTER INSERT. »

[ FAIRE ] Maintenant, change de filtre temporel **24h** → **1h** pour montrer que ça réagit.

[ DIRE ] « Si je passe la fenêtre de 24h à 1h, le dashboard se recalcule instantanément. »

[ FAIRE ] Bascule sur **Mongo Express** (onglet 2).

[ DIRE ] « Et voici la collection MongoDB en dessous. On retrouve les documents enrichis avec les sous-documents source, équipement, règle. C'est ce qui permet au dashboard d'être aussi rapide. »

[ FAIRE ] Bascule sur **PgAdmin / DBever** et exécute (depuis demo.sql) :

```
CALL rapport_risque_equipements();
```

[ DIRE ] « Et juste pour montrer une procédure stockée avec boucle FOR : voici le rapport de risque par équipement. La boucle parcourt tous les équipements et affiche le nombre d'alertes et le score moyen. »

[ TIMING ] Si tu manques de temps, saute le PgAdmin et reviens aux slides.

## Slide 12 — Conformité CDC (1 min)

[ FAIRE ] Slide 12.

[ DIRE ] « Pour conclure, voici le tableau de conformité au cahier des charges. Côté Projet A — PostgreSQL — j'ai validé les 6 critères pour 12 sur 12. Côté Projet B — MongoDB — j'ai validé les 7 types de requêtes pour 8 sur 8. Le total est 20 sur 20 et tous les critères sont remplis. »

### Slide 13 — Difficultés et perspectives (1 min)

[ FAIRE ] Slide 13.

[ DIRE ] \*« Honnêtement, j’ai rencontré quatre difficultés. La première : le choix entre normalisation côté PostgreSQL et enrichissement côté MongoDB. La solution finale combine les deux. La deuxième : l’ordre des triggers BEFORE et AFTER pour éviter les conflits. La troisième : la sérialisation des types INET et NUMERIC vers MongoDB. La quatrième : le setup Docker sous Windows 11 avec WSL2.

Pour les perspectives d’amélioration, j’ai déjà implémenté en bonus le streaming temps réel avec LISTEN et NOTIFY. Les prochaines étapes seraient un module de détection d’anomalies en machine learning avec IsolationForest, la géolocalisation des IP attaquantes sur une carte mondiale, une API REST FastAPI pour l’intégration avec un SOAR, et la Row Level Security pour limiter les analystes JUNIOR à leurs propres incidents. »\*

### Slide 14 — Merci (15 sec)

[ FAIRE ] Slide 14.

[ DIRE ] « Voilà, je vous remercie pour votre attention. Je suis prêt à répondre à vos questions. »

---

## 4. Démo de secours (en cas de problème technique)

Si Docker plante ou que le dashboard ne se lance pas le jour J :

1. **Reste calme.** Ne dis pas « ça marchait hier ».
  2. Bascule sur le PPTX et utilise **les captures de la slide 11.**
  3. Dis : « *Pour les besoins de la soutenance, j’utilise les captures du dashboard que j’ai prises hier soir. La logique est strictement la même, je peux vous montrer le code source juste après.* »
  4. Bascule alors sur **VS Code** et ouvre :
    - 01\_schema.sql (montre les CREATE TABLE)
    - 03\_triggers.sql (montre le trigger phare)
    - 08\_mongodb\_requetes.js (montre les 7 requêtes)
    - 10\_dashboard.py (montre l’intégration)
- 

## 5. Questions probables du prof — et tes réponses

### Q1. Pourquoi avoir utilisé les deux SGBD ensemble ? Un seul ne suffit pas ?

**Réponse type :** « Si, techniquement on peut tout faire avec un seul. Mais chacun a ses forces. PostgreSQL est imbattable pour l’intégrité transactionnelle — un incident de sécurité ne doit JAMAIS être perdu ou dédoublé, donc il faut ACID. En revanche, pour afficher 350 alertes dans un dashboard avec leurs sous-documents équipement et règle, MongoDB est plus efficace : pas de jointure, lecture en  $O(1)$ . Dans la vraie vie, Splunk fait exactement ça : la base maître est relationnelle et la couche d’affichage est NoSQL. »

### Q2. C’est quoi la différence entre une vue et une vue matérialisée ?

**Réponse type :** « Une vue est juste une requête sauvegardée. Quand on l’interroge, PostgreSQL exécute la requête sous-jacente. Donc c’est toujours à jour, mais ça peut être lent si la requête est complexe. Une vue matérialisée stocke physiquement le résultat sur disque

— c'est comme une cache. C'est très rapide à lire mais c'est figé : il faut faire `REFRESH MATERIALIZED VIEW` pour mettre à jour les données. Dans mon projet, j'utilise une vue matérialisée pour les KPI du dashboard parce qu'on les lit beaucoup et on tolère un léger décalage. »

### Q3. Pourquoi le type `INET` au lieu de `VARCHAR` ?

**Réponse type :** « Trois raisons. Un, `INET` valide automatiquement le format. Si je tente d'insérer "999.999.999.999", ça lève une erreur — ce qui n'est pas le cas avec `VARCHAR`. Deux, `INET` supporte les opérateurs réseau natifs : par exemple, je peux écrire `WHERE ip << '192.168.0.0/16'` pour trouver toutes les IP du réseau interne. Trois, `INET` prend moins de place que `VARCHAR(45)`. »

### Q4. Pourquoi avoir mis un index `GIN` sur le payload `JSONB` ?

**Réponse type :** « Parce que le payload contient des données très variables — l'URL attaquée, le nombre de tentatives, les ports scannés, etc. Sans index, une recherche du type `payload->>'url' LIKE '/api/login'` fait un Seq Scan sur toute la table. Avec un index `GIN`, la recherche se fait en quelques millièmes de seconde. Et c'est précisément ce qui rend PostgreSQL aussi performant que MongoDB sur ce genre de requête. »

### Q5. Comment marche ta chaîne de hashes dans `journal_audit` ?

**Réponse type :** « C'est inspiré de la blockchain. À chaque ligne d'audit, je calcule un `SHA-256` qui inclut le hash de la ligne précédente. Donc si quelqu'un modifie une ligne ancienne, tous les hashes suivants deviennent incohérents. J'ai même une fonction `verifier_integrite_audit()` qui recalcule la chaîne entière et retourne le numéro de la première ligne falsifiée. C'est pour démontrer un usage avancé de `pgcrypto`. »

### Q6. C'est quoi `LISTEN` / `NOTIFY` ?

**Réponse type :** « C'est le mécanisme natif de PostgreSQL pour faire du `publish / subscribe` entre la base et des clients externes. Côté trigger, on écrit `pg_notify('canal', message)`. Côté Python, le client fait `LISTEN canal` et reçoit le message en moins d'une seconde. Dans mon projet, ça remplace l'ETL batch : dès qu'une alerte est insérée, le dashboard MongoDB est mis à jour quasi instantanément. C'est plus élégant que de faire un polling toutes les 5 secondes. »

### Q7. Pourquoi pas SQL Server ou Oracle ?

**Réponse type :** « Trois raisons. Un, PostgreSQL est open-source et gratuit, donc reproductible. Deux, il a tous les types avancés dont j'avais besoin : `INET`, `JSONB`, `GIN`, `full-text`. Trois, c'est ce qu'utilisent en production les grosses boîtes de cybersécurité comme CrowdStrike, GitLab, ou Cloudflare. C'est aligné avec ma spécialisation. »

### Q8. Comment tu déploierais ce système en production ?

**Réponse type :** « Plusieurs étapes. Un, je remplacerais le payload `JSONB` par un stockage colonne style TimescaleDB ou ClickHouse pour la performance sur des milliards de lignes. Deux, je passerais MongoDB en réplica set 3 nœuds pour la haute disponibilité. Trois, j'ajouterais un Kafka entre les sondes et PostgreSQL pour absorber les pics de 100 000+

événements par seconde. Quatre, je sécuriserais avec TLS, secrets Vault et Row Level Security. Cinq, je mettrais le dashboard derrière un SSO Keycloak. »

**Q9. Tu as testé avec combien de données ?**

**Réponse type :** « Pour la soutenance, 350 alertes sur 7 jours. Mais j'ai écrit un script `seed_realistic.py` qui peut générer n'importe quel volume avec une distribution réaliste. Je l'ai testé jusqu'à 50 000 alertes et le dashboard reste fluide grâce aux index. »

**Q10. Si je te demande de présenter ça à un DSI non technique, comment tu fais ?**

**Réponse type :** « Je dirais : ce système permet à un analyste cybersécurité de détecter une attaque en moins d'une minute, là où il en mettrait des heures à éplucher des logs séparés. Concrètement, on évite à l'entreprise les 50 000 à 500 000 euros de dommages par incident moyen. Le coût d'implémentation est minime — quelques jours-homme — parce que c'est basé sur des composants open-source. »

---

## 6. Erreurs à éviter absolument

- NON Ne dis JAMAIS « c'est un projet d'école ». Présente-le comme un produit.
  - NON Ne lis JAMAIS les slides mot à mot. Regarde le prof, parle naturellement.
  - NON Ne montre PAS le code en gros pendant la présentation. Le code est dans le rapport.
  - NON Ne fais PAS de démo de plus de 3 minutes. Le prof veut voir, pas attendre.
  - NON Ne dis PAS « j'ai utilisé l'IA pour... ». Le prof veut savoir ce que tu sais.
  - NON Si tu ne sais pas, dis « je n'ai pas exploré ce point, c'est une bonne piste pour la v3 ».
- 

## 7. Cheat sheet — chiffres et termes à retenir

| Élément                   | Chiffre / Terme                    |
|---------------------------|------------------------------------|
| Tables PostgreSQL         | <b>6</b>                           |
| Séquences                 | <b>6</b>                           |
| Fonctions PL/pgSQL        | <b>6</b>                           |
| Triggers                  | <b>4</b>                           |
| Procédures (avec boucles) | <b>5</b>                           |
| Vues + matérialisée       | <b>4 + 1</b>                       |
| Index                     | <b>16</b>                          |
| Champs MongoDB            | <b>19</b>                          |
| Documents MongoDB         | <b>350+</b>                        |
| Index Mongo               | <b>5</b>                           |
| Score auto                | <b>0–100</b>                       |
| Seuil incident            | <b>5 alertes critiques / heure</b> |
| Latence streaming         | <b>&lt; 1 seconde</b>              |
| Total CDC                 | <b>20 / 20</b>                     |

**Mots-clés à utiliser :** ACID, CDC, KPI, NoSQL, JSONB, GIN, INET, pgcrypto, LISTEN/NOTIFY, \$lookup, \$facet, \$group, RGPD, SOC, SIEM, brute-force, DDoS, SQL injection.

---



## 8. Ouverture — le mot de la fin

Si tu termines en avance et que tu vois le prof attentif, conclus avec :

*« Ce qui m'intéresse particulièrement dans ce projet, c'est que ce n'est pas une simulation : un vrai SIEM en entreprise fonctionne exactement comme ça. La différence, c'est l'échelle — on parle de milliards d'événements par jour au lieu de 350. Mais l'architecture, les triggers, la séparation PostgreSQL / MongoDB, c'est identique. J'aimerais beaucoup approfondir ce sujet en stage de fin d'année. »*

C'est une **fin parfaite** pour un étudiant en spé cybersécurité.

---

**Bon courage. Tu connais ton sujet. Maintenant, montre-le.**

— Yassir